MiniCPM5-2B on Jetson Orin Nano 8 GB

Full benchmark report: 64K baseline A-D suite, DSpark speculative decoding, and a maxed-out 131,072-token configuration. Tested 2026-10-12/13 on a Jetson Orin Nano 8 GB (JetPack R36.4.7, aarch64) with the GUI stopped, using the standard 34-prompt A-D suite (cap profile v3, temp 0.3 / top_p 0.95 / top_k 40, exec_check code executed not inspected).

Model: MiniCPM5-2B (OpenBMB, Apache 2.0) - 2.52B dense, LlamaForCausalLM, 42 layers, 2 KV heads (8:1 GQA), native context 131,072. Target quant Q4_K_M (1.56 GB). DSpark draft: 323.8M params, 5 layers, dflash arch, Q4_K_M (186 MB).

1. Executive Summary

Metric	Result
Overall quality (34 prompts, 64K run)	6.53 / 10
Generation speed, no drafter (64K)	25.3 tok/s avg (peak 27.2)
Generation speed, DSpark (suite A)	52.3 tok/s avg - 2.0x
Peak generation (a_code, DSpark)	84.3 tok/s - fastest measured on this box
Deep needle recall	PASS - exact PINECONE-42 at 121,426-token depth
Memory at 64K q8_0 KV	3.41 GB RSS (leanest 64K fit tested)
Memory at 131K q8_0 KV + DSpark	5.74 GB RSS (fits with GUI off)
exec_check (code executed)	3/3 pass (b_python, c_python, d_fib)
Function calls	5/6 (one refusal)
Hard traps (grade >= 7)	6/12

Verdict in one line: MiniCPM5-2B is not a Qwen3.5-2B replacement for the resident agent slot on raw quality, but with DSpark it is the fastest 2B-class generation engine tested on this hardware, and its 2-KV-head architecture makes it the only model that can run a full 128K window on an 8 GB Jetson.

2. Test Configurations

2.1 Baseline run (64K)

Parameter	Value
Server	llama.cpp 0.6.0-dev (build 1098), CUDA, -ngl 99, -fa on, --jinja, -t 6
Context / KV	-c 65536, -ctk q8_0 -ctv q8_0 (1.41 GB KV)
Batch	-b 512 -ub 512, single slot
Thinking	OFF via chat_template_kwargs {enable_thinking: false}
Caps (profile v3)	A 6500, B 24000, C 4096 (func 1024), D 3072
Load	6.0 s to healthy; 3.41 GB RSS steady

2.2 Maxed run (131K + DSpark)

Parameter	Value
Context / KV	-c 131072 (full native), -ctk q8_0 -ctv q8_0 (2.82 GB KV)
Draft model	-md DSpark Q4_K_M, --spec-type draft-dspark, --spec-draft-n-max 7, -ngld 99
Draft KV	-ctkd q8_0 -ctvd q8_0 (167 MB)
Batch	-b 256 -ub 256 (reduced from 512 to fit draft compute buffers)
Output headroom	--predict -1 (unlimited; requests bounded only by context)
Load	8.0 s to healthy; 5.74 GB RSS; 8.0 GB total DRAM, GUI off

Note: the first maxed launch OOM-killed the draft's compute buffer at -ub 512 (272 MiB cudaMalloc failure). Dropping to -ub 256 and the Q4_K_M draft resolved it.

3. Speed Results

3.1 Baseline, no drafter (64K, 34 prompts)

Suite	n	Avg gen tok/s	Range	Avg completion tokens	Capped
A - general	5	25.95	25.2 - 26.5	616	0/5
B - one-shot coding	5	25.59	25.1 - 26.1	1040	0/5
C - zoo/ornith	12	23.97	18.3 - 26.6	272	0/12
D - reasoning traps	12	26.30	25.7 - 27.2	341	0/12
Overall	34	25.32	18.3 - 27.2	-	0/34

Prompt processing averaged 431 tok/s; TTFT averaged 0.14 s. Tool-call bursts (17-46 tokens) dipped to 18-19 tok/s. Zero truncations across all 34 prompts.

3.2 DSpark speculative decoding (suite A, apples-to-apples)

Prompt	No drafter tok/s	DSpark tok/s	Speedup
a_code	25.96	84.27	3.25x
a_iambic	26.51	42.54	1.60x
a_math	25.93	63.27	2.44x
a_prose	25.20	40.32	1.60x
a_creative	26.16	31.10	1.19x
Suite A average	25.95	52.30	2.02x

Acceptance behavior (from server logs): 16.4% acceptance cold on the first generation, warming to 39-43% with mean accepted draft length 3.99 of the 7-token block. Content matters: code (repetitive syntax) reached 84.3 tok/s (3.2x) while freeform prose reached 31-40 tok/s (1.2-1.6x). Quick probes: essay 31.0, code 57.4, short reasoning 53.8 tok/s.

Draft overhead is negligible in memory (186 MB weights + 167 MB KV). DSpark is distributionally lossless: the target model still decides every accepted token. Temp-0.3 outputs differ between runs (sampling noise, not draft artifacts).

4. Quality (64K suite, graded 1-10 vs grade_key)

Suite	Avg grade	Highlights
A - general	7.00	sqrt(2) proof 10/10, clinical prose 8/10, iambic pentameter 2/10
B - coding	6.80	b_python executed+correct 9/10; TRS-80 BASIC 3/10
C - zoo/ornith	7.25	induction proof 9/10, c_python exec pass 9/10; sqlite refusal 0/10
D - traps	5.50	batball 10, sum100 10, sqrt2 9, fib 10; strawberry 2, elephant 1
Overall	6.53	34 prompts

4.1 Objective verification (exec_check: executed, not inspected)

Check	Result
b_python - employee data processing	PASS - runs clean; correct dept averages (HR 75k / Eng 90k), top-3 output
c_python - student grade processor	PASS - runs clean; avg 86.25, top Diana, bottom Charlie
d_fib - memoized fibonacci	PASS - verified programmatically: fib(10)=55, fib(30)=832040 in <0.1 s

4.2 Function calling (C-suite)

Prompt	Expected	Got	Result
c_func_web_search	web_search	web_search(query)	OK
c_func_terminal	terminal	terminal('df -h /')	OK
c_func_write_file	write_file	write_file(path, content)	OK
c_func_read_file	read_file	read_file(path)	OK
c_func_email	none (draft in text)	clean text draft, no call	OK
c_func_sqlite	terminal	REFUSAL: 'unable to query databases'	FAIL

Tool names and arguments were correct on every call attempted. The single failure is a guardrail-style refusal ('...or access patient information') on a benign agentic task - a training artifact, not a capability gap.

5. Deep Needle at Full Window (121K)

Needle construction identical to the LFM2.5 runbook: needle sentence at position ~0, 7,140 repetitions of the logistics filler, question last. Filler packing measured on MiniCPM's own tokenizer: 17.01 tokens/line (matching LFM's 17.0, versus ~26 on Qwen-class tokenizers). Greedy decoding (temp 0).

Metric	Value
Grade	PASS
Depth (actual usage.prompt_tokens)	121,426 tokens
Answer	contains exact PINECONE-42 (rambling phrasing, 32-token cap)
Prompt processing at depth	271.7 tok/s (446.9 s to ingest)
Total wall time	453.4 s

Interpretation: exact-token recall across a 121K window, inside the model's 131K native training boundary. This mirrors LFM2.5's pattern - exact recall within native context, degradation beyond it (LFM dropped to 'PINECONE' at 49.3K vs its 32K native). MiniCPM5-2B is the first model tested on this box whose native window makes a 128K deployment meaningful, and the first to pass a needle at >120K depth on 8 GB.

6. Findings

F1 - Letter-counting blind spot

Both counting traps failed with confident, wrong enumeration: '4 r's' in strawberry (a phantom 4th r) and '3 e's' in elephant (a hallucinated third e). Qwen3.5-2B passes both. Treat letter/character-counting tasks as out-of-scope for this model.

F2 - Refusals on benign tasks

2 of 34 prompts refused benign requests: the sqlite function call and 'why does ice float' ('cannot answer without context'). Also failed the piano tuners Fermi estimate by treating it as a joke (answered zero). Over-tuned guardrails directly reduce agentic reliability.

F3 - Stochastic repetition pathology

One strawberry smoke run looped 'but wait...' re-verification to the 3072-token cap despite a correct early answer (9 'wait' markers); the recorded run miscounted instead of looping. Both iambic prompts degenerated into repeated phrases. Repeat_penalty 1.0 (standard) was used; watch long generations for loops.

F4 - Thinking is on by default

Out of the box the model streams reasoning (63 tokens on a trivial question; a full 3072-cap burn on one trap). The embedded switch enable_thinking:false works cleanly through llama.cpp chat_template_kwargs and matches the Qwen3.5-2B graded configuration.

F5 - Memory and cursor champion

2 KV heads give 43 KB/token KV cost: 1.41 GB at 64K, 2.82 GB at 131K (q8_0). The 3.41 GB RSS at 64K is the leanest ever measured here (LFM2.5: 4.84 GB), and the 131K full-window load fits 8 GB with the GUI off.

F6 - DSpark transforms speed

2.0x on suite A average (52.3 vs 25.95 tok/s), up to 3.2x on code (84.3 tok/s - the fastest generation ever measured on this Jetson, versus Qwen3.5-2B's MTP peak of 46.6). Draft costs only 353 MB total.

7. Model Comparison (same suite, caps, sampling)

Metric	Qwen3.5-2B	LFM2.5-VL-3B	MiniCPM5-2B	MiniCPM5 MAXED
Config	Q4_K_M, MTP, q8 KV	Q4_K_M + DSpark, f16 KV, YaRN	Q4_K_M, no drafter	131K ctx + DSpark
Gen tok/s	37.0 avg (peak 46.6)	35.2 avg	25.3 avg (peak 27.2)	52.3 avg (peak 84.3)
Quality	7.4 / 10	-	6.5 / 10	identical (lossless)
Hard traps	4 / 4	5 / 5	3 / 4	same
Function calls	6 / 6	5 / 6	5 / 6 (1 refusal)	same
RSS @ 64K	~4.3 GB	4.84 GB	3.41 GB	5.74 GB @ 131K

8. Verdict and Recommendations

As the resident Hermes-agent model: NO. Qwen3.5-2B remains the right choice - higher quality (7.4 vs 6.5), faster with MTP at short agentic generations, and it passes the traps MiniCPM5 misses. MiniCPM5's two refusals on benign tasks are disqualifying for an unattended agent loop.

Where MiniCPM5-2B wins: (1) long-context workloads - it is the only model that holds a true 128K window on 8 GB, with verified exact recall at 121K; (2) bulk generation with DSpark - 2x on prose, 3.2x on code, at a trivial memory cost; (3) memory-constrained deployments needing headroom alongside other services.

Recommended follow-ups: (a) rerun the interrupted max-token probe (32K-token long-form generation) to characterize the repetition loop at length; (b) a greedy A/B with and without the draft to document strict losslessness; (c) try --spec-draft-n-max beyond 7 (the draft was trained with block 7, but the scheduler may accept longer chains on code).

Outstanding item from the maxed session: the 32,768-token max-output probe was interrupted by a gateway shutdown mid-generation (server-side connection drop, no results recorded). Everything else in this report is backed by saved raw data in the project directory.

Appendix A - Methodology

Suite: canonical 34-prompt A-D suite (prompts_abcd.json, verbatim across all models): A general x5, B one-shot coding x5 (HTML/Python/C/TRS-80 BASIC/Julia), C zoo/ornith x12 (incl. 6 function-call with tool schemas), D reasoning traps x12. Cap profile v3: A 6500, B 24000 (deep-budget probe), C 4096 with function-calls at 1024, D 3072. Sampling: temp 0.3, top_p 0.95, top_k 40, min_p 0.0, repeat_penalty 1.0, streaming. exec_check code was executed, never inspected. Needle tests greedy (temp 0). Hard traps graded >= 7/10.

Harnesses: bench.py (live-server streaming, resume-safe JSONL, per-prompt ttft/pp/gen tok/s) adapted from the qwen35-2b benchmark; needle_128k.py (filler sized via /tokenize on the model's own tokenizer). Launches detached via Popen(start_new_session=True) per the long-batch runbook; the GUI (gdm3) was stopped for all runs. Raw data: results/bench_minicpm5_20261012.jsonl (34 rows), results/bench_minicpm5_131k_dspark_A.jsonl (5 rows), results/needle_121k.json, results/grades_minicpm5_20261012.json, results/responses/ (34 transcripts), results/exec/ (executed code + outputs).

Appendix B - Full 34-Prompt Results (64K baseline run)

ID	Suite	Grade	CT	gen tok/s	Finish
a_code	A	8	446	26.0	stop
a_creative	A	7	331	26.2	stop
a_iambic	A	2	87	26.5	stop
a_math	A	10	648	25.9	stop
a_prose	A	8	1566	25.2	stop
b_basic	B	3	349	26.1	stop
b_c	B	7	941	25.7	stop
b_html	B	8	1701	25.1	stop
b_julia	B	7	640	25.9	stop
b_python	B	9	1568	25.2	stop
c_creative	C	7	302	26.2	stop
c_func_email	C	8	56	26.6	stop
c_func_read_file	C	10	19	18.9	tool_calls
c_func_sqlite	C	0	26	26.4	stop
c_func_terminal	C	10	17	18.3	tool_calls
c_func_web_search	C	10	23	19.0	tool_calls
c_func_write_file	C	9	46	22.1	tool_calls
c_html_game	C	7	978	25.6	stop
c_html_profiles	C	7	751	25.8	stop
c_iambic	C	1	90	26.5	stop

ID	Suite	Grade	CT	gen tok/s	Finish
c_math_proof	C	9	455	26.1	stop
c_python	C	9	502	26.0	stop
d_batball	D	10	206	26.4	stop
d_binsearch	D	8	943	25.7	stop
d_boxes	D	4	773	25.8	stop
d_e_elephant	D	1	93	26.5	stop
d_fib	D	10	192	26.4	stop
d_ice	D	1	19	27.2	stop
d_piano	D	1	111	26.5	stop
d_r_strawberry	D	2	135	26.4	stop
d_sheep	D	7	329	26.2	stop
d_sqrt2	D	9	911	25.8	stop
d_sum100	D	10	142	26.4	stop
d_switches	D	3	240	26.3	stop